

ReleaseTrack Dashboard 설계 문서

DB를 연동한 웹 서비스 **ReleaseTrack Dashboard**의 설계 문서입니다. 과제 요건의 설계 문서 필수 항목을 순서대로 정리했습니다.

Spring Boot 3

React 18 · TypeScript

H2 Database

REST API

01 서비스 개요

무엇을 만들었는지

ReleaseTrack Dashboard는 여러 서비스의 배포 이력, 배포 상태, 변경사항, 실패 사유를 한 화면에서 관리하기 위한 내부 운영 대시보드입니다.

서비스 운영 과정에서는 배포 결과와 변경사항이 CI/CD 로그, Git 저장소, 이슈 관리 도구, 메신저 등에 흩어지는 경우가 많습니다. 이 프로젝트는 배포 이력을 별도 데이터로 구조화해 저장하고, REST API와 React 화면을 통해 조회·등록·수정·삭제할 수 있도록 구현했습니다.

주요 기능은 다음과 같습니다.

- 서비스 목록 조회, 등록, 상세 조회, 수정
- 배포 이력 목록 조회, 등록, 상세 조회, 수정, 삭제
- 배포 변경사항 관리
- 최근 7일 배포 추이와 상태별 통계 확인
- 서비스별 마지막 배포 상태 확인
- 서비스별 환경 파이프라인 확인

왜 이 주제를 선택했는지

과제가 요구하는 기술 요소를 한 도메인 안에 자연스럽게 담기에 배포 이력 관리가 적합하다고 판단했습니다.

단순 CRUD 게시판보다 상태 전이(대기 → 진행 중 → 성공/실패/롤백), 통계 집계, 환경 파이프라인 흐름 같은 실질적인 로직을 보여줄 수 있고, 배포·릴리스라는 도메인은 개발자에게 익숙해 설계 의도를 직관적으로 전달하기 좋다고 보았습니다.

다만 과제 범위에서는 실제 Jenkins, GitHub Actions, GitLab CI Webhook 연동은 제외하고, **수동 등록**이 가능한 **배포 이력 관리 시스템**으로 구현했습니다.

02 기술 스택 선택 근거

Backend

구분	선택 기술
Language	Java 17
Framework	Spring Boot 3.3.x
Build Tool	Gradle
ORM	Spring Data JPA
DB	H2 Database
Validation	Jakarta Bean Validation
Test	Spring Boot Test, MockMvc

Java 17은 Spring Boot 3에서 안정적으로 사용할 수 있는 LTS 버전이고, 과제 요구사항과도 맞습니다. Spring Boot는 REST API 서버를 빠르게 구성할 수 있고 Controller·Service·Repository 계층을 명확히 나누기 좋아 선택했습니다.

Spring Data JPA는 서비스·배포·변경사항처럼 관계가 명확한 도메인을 Entity로 표현하기 적합합니다. CRUD와 페이지는 Repository로 처리하고, 대시보드 집계처럼 화면 요구사항이 있는 로직은 Service 계층에서 조합했습니다.

H2는 별도 DB 설치 없이 로컬에서 실행할 수 있어 과제 리뷰 환경에 적합합니다. 운영 환경으로 확장한다면 PostgreSQL 같은 RDBMS를 우선 고려할 수 있습니다.

Frontend

구분	선택 기술
Framework	React 18
Language	TypeScript
Build Tool	Vite
Routing	React Router
Server State	TanStack Query
HTTP Client	Axios
Chart	Recharts
UI Primitive	Radix UI 기반 로컬 컴포넌트
Test	Vitest, Testing Library

React는 대시보드·목록·상세·폼 화면을 컴포넌트 단위로 나누기 좋고 과제 요구사항과도 맞습니다. TypeScript는 API 응답 타입, enum 상태값, 폼 요청 타입을 명확히 관리하기 위해 사용했습니다.

Vite는 개발 서버 실행과 빌드가 빠르고 설정이 단순합니다. TanStack Query는 서버 상태 조회, mutation 이후 캐시 무효화, 로딩/에러 상태 처리를 일관되게 만들기 위해 선택했습니다.

UI는 shadcn/ui(Radix + Tailwind) 방식으로 `src/components/ui` 에 로컬 컴포넌트를 구성했습니다. 완성형 프레임워크의 기본 스타일에 얽매이지 않고 대시보드 톤과 컴포넌트 스타일을 디자인 방향에 맞춰 세밀하게 제어하기 위해서입니다.

DB

개발 및 제출 환경에서는 H2를 사용했습니다. 별도 설치 없이 `./gradlew bootRun` 만으로 API와 샘플 데이터를 확인할 수 있기 때문입니다.

전체 시스템 흐름

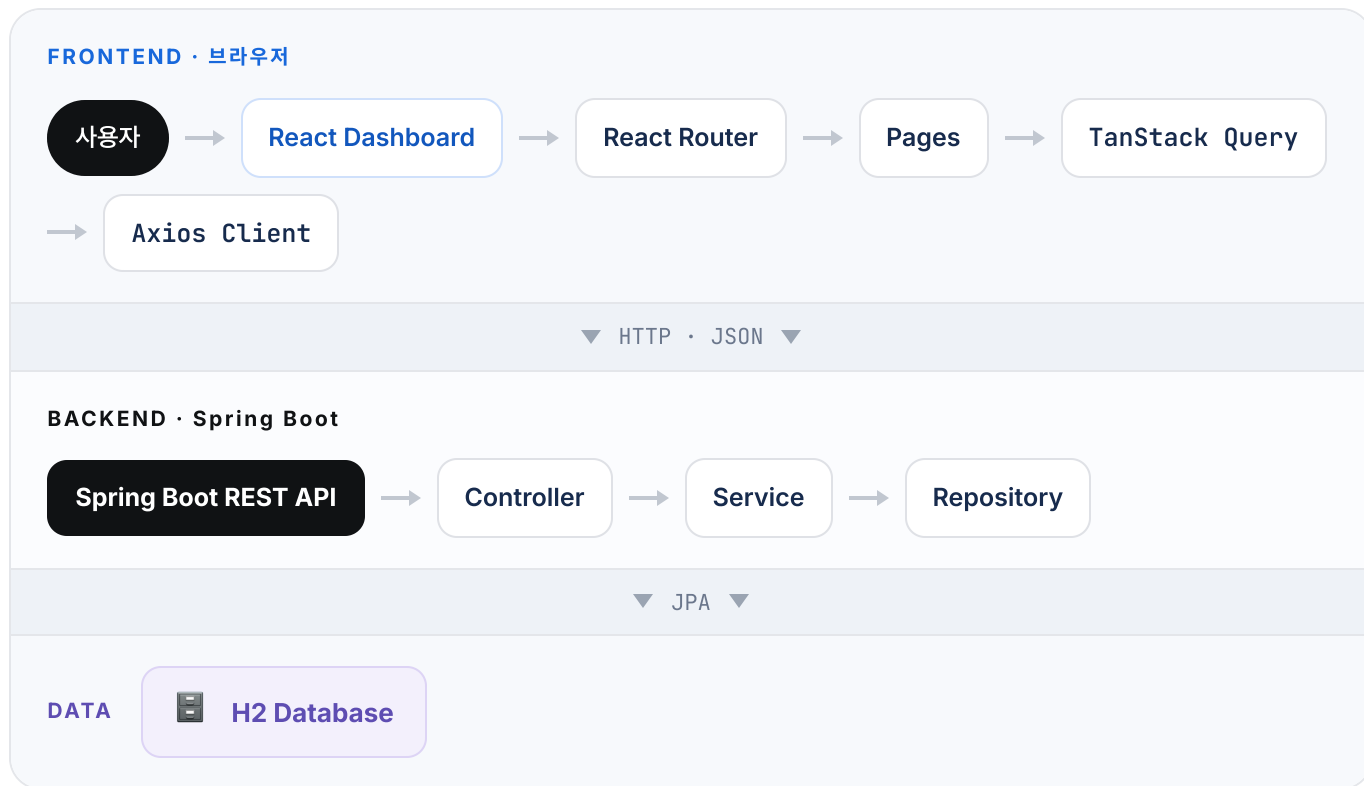


그림 1. 요청은 프론트엔드 → 백엔드 계층 → H2 순으로 흐른다.

사용자는 React 대시보드에서 서비스와 배포 이력을 관리하며, React Router가 화면 경로를 처리합니다. 화면 컴포넌트는 TanStack Query hook을 통해 API 데이터를 조회하거나 변경하고, Spring Boot는 Controller·Service·Repository 계층으로 요청을 처리해 H2 Database에 저장합니다.

컴포넌트 관계



그림 2. App 셀의 3개 메뉴 라우트 기준 컴포넌트 구성.

- `DashboardHome` 은 데이터 조회와 블록 레지스트리(표시/순서 개인화)만 담당하는 셸이고, 각 블록은 별도 컴포넌트로 분리되어 있습니다.
- 차트 카드(`TrendCard` · `StatusDistCard`)는 `DashboardCharts` 로, 파이프라인 카드는 `PipelineCard` 로 분리했습니다. 공통 툴팁(`ChartTooltip`)과 카드 헤더 선택(`CardControlSelect`)는 재사용합니다.
- 표 계열 블록은 `DashboardBlock` 래퍼(제목 헤더 + 본문)로 감싸 보일러플레이트를 제거했습니다.
- `localStorage` 접근은 `lib/storage` 의 `safeStorage` 와 `useLocalStorageState` 혹으로 일원화해, 레이어아웃·파이프라인 선택·알림 숨김 상태를 안전하게 지속합니다.

요청 흐름

요청
 응답
 ↻ 내부 처리

① 배포 이력 목록 조회

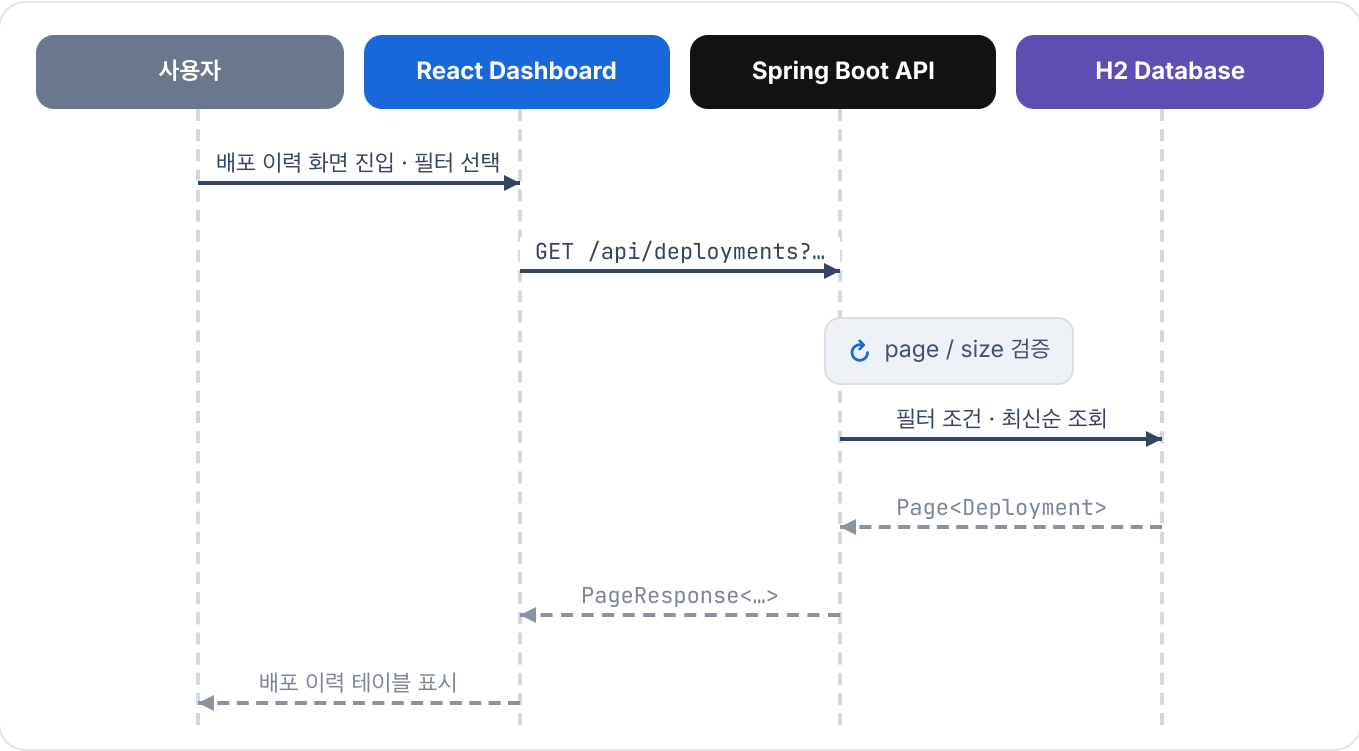


그림 3. 배포 이력 목록 조회 시퀀스.

② 배포 이력 등록

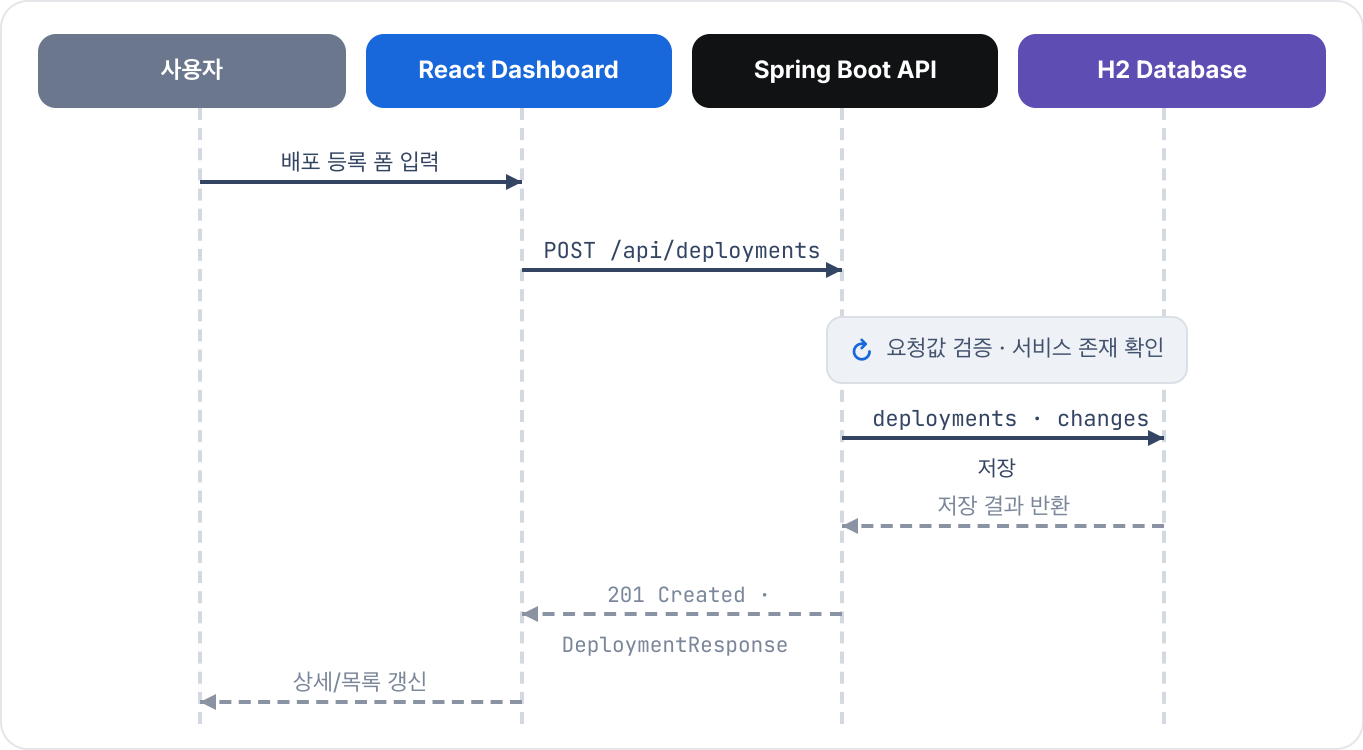


그림 4. 배포 이력 등록 시퀀스.

③ 대시보드 요약 조회

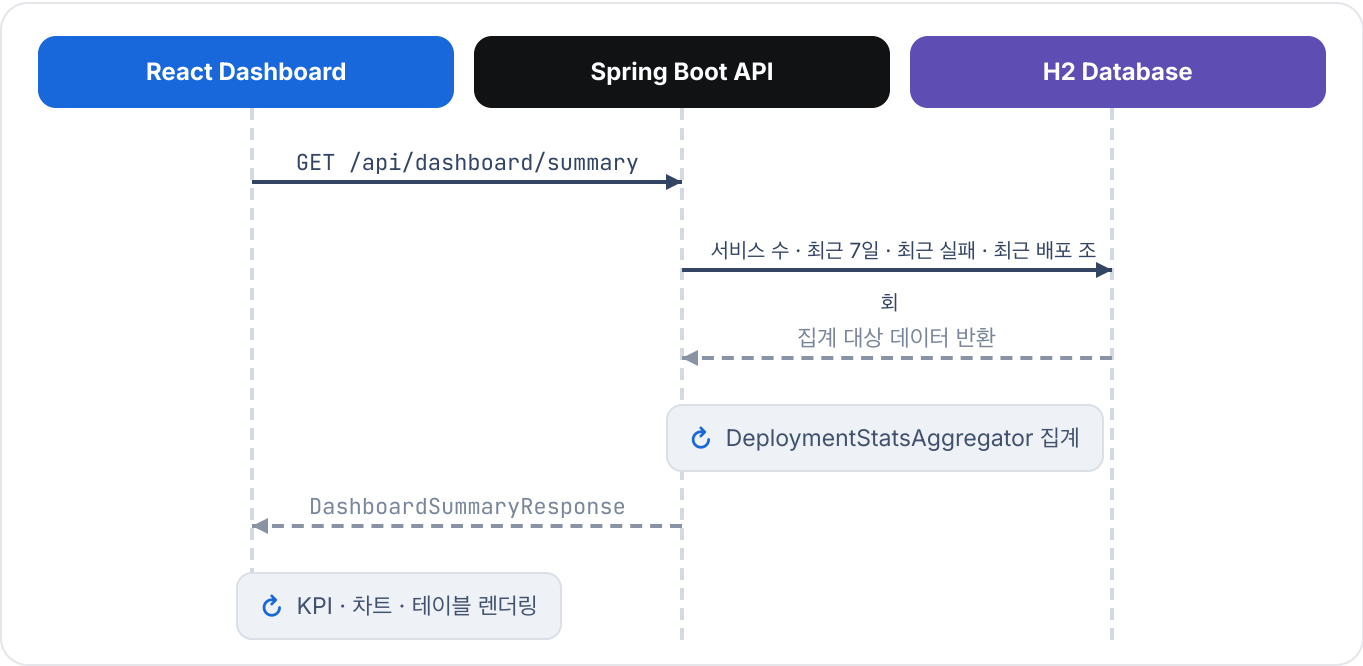


그림 5. 대시보드 요약 조회 시퀀스.

04 DB 설계

ERD

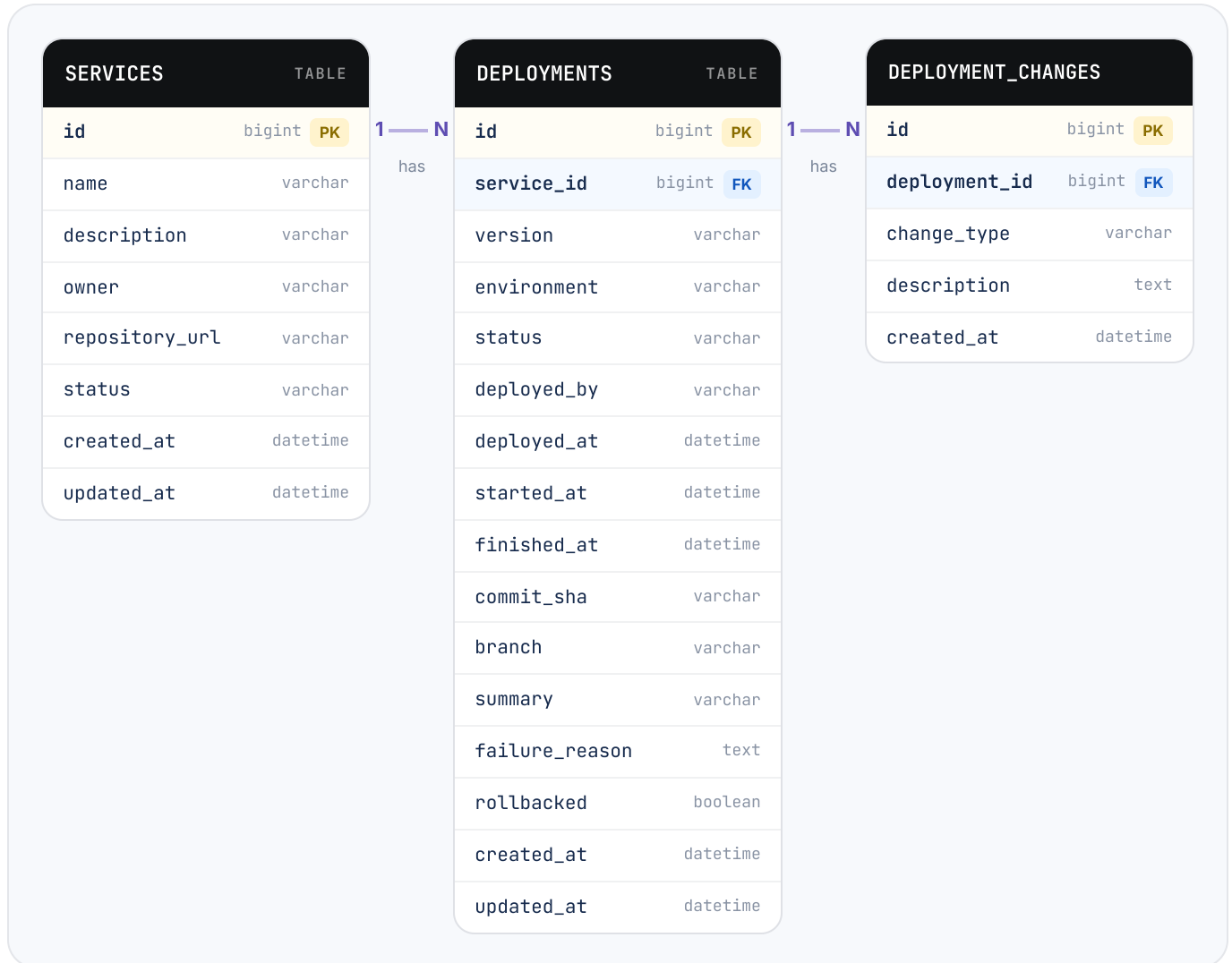


그림 6. SERVICES → DEPLOYMENTS → DEPLOYMENT_CHANGES 1:N 관계. PK 기본키 · FK 외래키.

테이블 정의

services

컬럼	타입	제약	설명
id	BIGINT	PK, Auto Increment	서비스 ID
name	VARCHAR(100)	NOT NULL	서비스명
description	VARCHAR(500)	NULL	서비스 설명
owner	VARCHAR(100)	NULL	담당 팀/담당자
repository_url	VARCHAR(500)	NULL	저장소 URL
status	VARCHAR(30)	NOT NULL	ACTIVE · ARCHIVED · MAINTENANCE
created_at	DATETIME	NOT NULL	생성일
updated_at	DATETIME	NOT NULL	수정일

deployments

컬럼	타입	제약	설명
id	BIGINT	PK, Auto Increment	배포 이력 ID
service_id	BIGINT	FK, NOT NULL	서비스 ID
version	VARCHAR(50)	NOT NULL	배포 버전
environment	VARCHAR(30)	NOT NULL	DEV · STAGING · PRODUCTION
status	VARCHAR(30)	NOT NULL	QUEUED · RUNNING · SUCCESS · FAILED · ROLLED_BACK
deployed_by	VARCHAR(100)	NOT NULL	배포자
deployed_at	DATETIME	NOT NULL	배포 기준 시각
started_at	DATETIME	NULL	배포 시작 시각
finished_at	DATETIME	NULL	배포 종료 시각
commit_sha	VARCHAR(100)	NULL	배포 커밋 SHA
branch	VARCHAR(200)	NULL	배포 브랜치
summary	VARCHAR(500)	NULL	배포 요약
failure_reason	TEXT	NULL	실패/롤백 사유
rollbacked	BOOLEAN	NOT NULL	롤백 여부
created_at	DATETIME	NOT NULL	생성일
updated_at	DATETIME	NOT NULL	수정일

deployment_changes

컬럼	타입	제약	설명
id	BIGINT	PK, Auto Increment	변경사항 ID
deployment_id	BIGINT	FK, NOT NULL	배포 이력 ID
change_type	VARCHAR(30)	NOT NULL	FEATURE · BUG_FIX · REFACTOR · CONFIG · ETC
description	TEXT	NOT NULL	변경 내용
created_at	DATETIME	NOT NULL	생성일

DB 설계 의도

서비스와 배포 이력을 분리한 이유는 하나의 서비스에 여러 배포 이력이 시간순으로 누적되기 때문입니다. 서비스는 관리 대상의 현재 정보이고, 배포 이력은 이벤트성 데이터입니다.

배포 변경사항은 deployment_changes 로 분리했습니다. 하나의 배포에 여러 변경사항이 포함될 수 있고, 변경 유형을 분리하면 추후 변경 유형별 통계나 필터링으로 확장하기 쉽습니다.

started_at · finished_at · commit_sha · branch 는 배포 추적성을 높이기 위해 추가했습니다.
durationSec 는 DB에 저장하지 않고 started_at · finished_at 에서 파생해 응답 DTO에서 제공합니다.

인덱스 설계 및 이유

현재 `schema.sql` 기준 인덱스는 실제 조회 조건과 정렬 패턴을 기준으로 설계했습니다.

인덱스 · 컬럼	이유
idx_services_status_name status, name	서비스 목록은 status 필터와 name 오름차순 정렬을 함께 사용합니다. <code>countByStatus(status)</code> 에도 status 선두 인덱스가 유효합니다.
idx_services_name name	status 없이 전체 서비스 목록을 name 기준으로 정렬할 때 사용합니다.
idx_deployments_deployed_at_id deployed_at desc, id desc	최근 배포 10건, 최근 7일 집계, 배포 목록 기본 최신순 정렬을 지원합니다. id를 함께 뒤 동일 시각에서도 정렬이 결정적입니다.
idx_deployments_service_deployed_at_id service_id, deployed_at desc, id desc	특정 서비스의 최신 배포 조회와 serviceId 필터가 있는 배포 목록 조회를 지원합니다.
idx_deployments_service_environment_deployed_at_id service_id, environment, deployed_at desc, id desc	서비스 상세의 환경별 최신 배포 파이프라인 조회를 지원합니다.
idx_deployments_status_deployed_at_id status, deployed_at desc, id desc	최근 실패/롤백 배포 조회와 status 필터 목록 조회를 최신순 정렬까지 지원합니다.
idx_deployments_environment_deployed_at_id environment, deployed_at desc, id desc	environment 필터 목록 조회를 최신순 정렬까지 지원합니다.
idx_deployments_branch_deployed_at_id branch, deployed_at desc, id desc	branch 필터 목록 조회를 최신순 정렬까지 지원합니다.
idx_deployment_changes_deployment_id deployment_id	배포 상세 조회에서 변경사항 목록을 deployment_id로 조인할 때 사용합니다.

05 API 설계

설계 원칙

- URL은 리소스 중심으로 구성했습니다.
- 조회는 `GET` , 생성은 `POST` , 부분 수정은 `PATCH` , 삭제는 `DELETE` 를 사용했습니다.
- Entity를 직접 노출하지 않고 Response DTO로 변환합니다.

- 목록 응답은 프론트엔드에서 필요한 필드만 포함하고, 상세 응답은 변경사항과 실패 사유 등 상세 필드를 포함합니다.
- 요청 검증 실패는 `400` , 대상 리소스가 없으면 `404` 를 반환합니다.

주요 엔드포인트

Method	Path	설명
GET	/api/services	서비스 목록 조회. <code>status</code> · <code>keyword</code> 필터 지원
POST	/api/services	서비스 등록
GET	/api/services/{serviceId}	서비스 상세 조회
PATCH	/api/services/{serviceId}	서비스 수정
GET	/api/deployments	배포 이력 목록 조회. <code>serviceId</code> · <code>environment</code> · <code>status</code> · <code>branch</code> · <code>from</code> · <code>to</code> · <code>page</code> · <code>size</code> 지원
POST	/api/deployments	배포 이력 등록
GET	/api/deployments/{deploymentId}	배포 이력 상세 조회
PATCH	/api/deployments/{deploymentId}	배포 이력 수정
DELETE	/api/deployments/{deploymentId}	배포 이력 삭제
GET	/api/dashboard/summary	대시보드 요약 조회
GET	/api/dashboard/pipeline?serviceId={id}	서비스의 DEV/STAGING/PRODUCTION 최신 배포 상태 조회

HTTP 상태 코드

상태 코드	사용 상황
200 OK	조회, 수정 성공
201 Created	생성 성공
204 No Content	삭제 성공
400 Bad Request	요청값 검증 실패, page/size 범위 오류
404 Not Found	대상 서비스 또는 배포 이력 없음
500 Internal Server Error	처리하지 못한 서버 오류

요청/응답 예시

서비스 목록 조회

```
GET /api/services?status=ACTIVE&keyword=api

[
  {
    "id": 2,
    "name": "admin-api",
    "description": "관리자 API 서비스",
    "owner": "platform-team",
    "repositoryUrl": "https://github.com/example/admin-api",
    "status": "ACTIVE",
    "lastDeploymentVersion": "v2.0.1",
    "lastDeployedAt": "2026-07-01T10:00:00",
    "createdAt": "2026-07-01T09:00:00",
    "updatedAt": "2026-07-01T09:00:00"
  }
]
```

배포 이력 목록 조회

```
GET /api/deployments?environment=PRODUCTION&status=SUCCESS&page=0&size=20
```

```
{
  "content": [
    {
      "id": 1,
      "serviceId": 1,
      "serviceName": "customer-web",
      "version": "v1.2.3",
      "environment": "PRODUCTION",
      "status": "SUCCESS",
      "deployedBy": "mason",
      "deployedAt": "2026-07-01T11:00:00",
      "summary": "결제 화면 UI 개선 및 오류 수정",
      "rollbacked": false,
      "branch": "main"
    }
  ],
  "page": 0,
  "size": 20,
  "totalElements": 1,
  "totalPages": 1,
  "first": true,
  "last": true
}
```

배포 이력 등록

```
POST /api/deployments
Content-Type: application/json
```

```
{
  "serviceId": 1,
  "version": "v1.2.5",
  "environment": "PRODUCTION",
  "status": "SUCCESS",
  "deployedBy": "mason",
  "deployedAt": "2026-07-01T15:00:00",
  "startedAt": "2026-07-01T14:52:00",
  "finishedAt": "2026-07-01T15:00:00",
  "commit": "4b5c6d7e8f901234560102030405060708091011",
  "branch": "main",
  "summary": "주문 내역 화면 개선",
  "failureReason": null,
  "rollbacked": false,
  "changes": [
    { "changeType": "FEATURE", "description": "주문 내역 필터 기능 추가" },
    { "changeType": "BUG_FIX", "description": "상세 진입 시 로딩이 멈추던 문제 수정" }
  ]
}
```

배포 상세 응답

```
{
  "id": 1,
  "serviceId": 1,
  "serviceName": "customer-web",
  "version": "v1.2.3",
  "environment": "PRODUCTION",
  "status": "SUCCESS",
  "deployedBy": "mason",
  "deployedAt": "2026-07-01T11:00:00",
  "summary": "결제 화면 UI 개선 및 오류 수정",
  "rollbacked": false,
  "branch": "main",
  "failureReason": null,
  "commit": "a1b2c3d4e5f60718293a4b5c6d7e8f9012345601",
  "startedAt": "2026-07-01T10:53:00",
  "finishedAt": "2026-07-01T11:00:00",
  "durationSec": 420,
  "changes": [
    { "id": 1, "changeType": "FEATURE", "description": "결제 완료 화면에 안내 문구 추가" }
  ],
  "createdAt": "2026-07-01T09:00:00",
  "updatedAt": "2026-07-01T09:00:00"
}
```

대시보드 요약 조회

```
GET /api/dashboard/summary
```



```
{
  "totalServiceCount": 8,
  "activeServiceCount": 6,
  "weeklyDeploymentCount": 12,
  "productionDeploymentCount": 5,
  "successDeploymentCount": 7,
  "failedDeploymentCount": 2,
  "rollbackCount": 1,
  "successRate": 58.3,
  "deploymentTrendByStatus": {
    "series": ["SUCCESS", "FAILED", "RUNNING"],
    "points": [
      { "date": "2026-06-25", "SUCCESS": 1, "FAILED": 0, "RUNNING": 0 }
    ]
  },
  "statusStats": [
    { "status": "SUCCESS", "count": 7 },
    { "status": "FAILED", "count": 2 }
  ],
  "recentDeployments": [
    {
      "id": 1,
      "serviceName": "customer-web",
      "version": "v1.2.3",
      "environment": "PRODUCTION",
      "status": "SUCCESS",
      "deployedBy": "mason",
      "deployedAt": "2026-07-01T11:00:00",
      "summary": "결제 화면 UI 개선 및 오류 수정"
    }
  ]
}
```

검증 정책

서비스 등록/수정

- `name` 필수, 공백 불가, 최대 100자
- `description` 최대 500자
- `owner` 최대 100자
- `repositoryUrl` 최대 500자
- `status` ACTIVE · ARCHIVED · MAINTENANCE

배포 등록/수정

- `serviceId` 필수, 존재하는 서비스 ID
- `version` 필수, 최대 50자
- `environment` DEV · STAGING · PRODUCTION
- `status` QUEUED · RUNNING · SUCCESS · FAILED · ROLLED_BACK
- `deployedBy` 필수, 최대 100자 · `deployedAt` 필수
- `branch` 최대 200자 · `commit` 최대 100자
- `changes` 0개 이상, 각 `description` 필수

어떤 도구를 어떤 목적으로 사용했는지

도구	활용 목적
Claude Design	대시보드/목록/상세 화면의 디자인 초안 작성 — 레이아웃과 시각 방향을 빠르게 잡는 용도
ChatGPT / Codex	설계 문서 초안 작성, README 분리, 인덱스 설계 검토, 코드 구조 점검, Claude Code와의 크로스 체크
Claude Code	에이전트/플러그인 기반 개발 환경 — Harness·Analyst·Backend·Frontend·Code Reviewer·Integration QA를 모두 여기서 실행
Harness Plugin	역할별 작업 환경과 지시 세트를 빠르게 구축하고 프론트엔드/백엔드/QA/리뷰 작업 흐름을 분리해 진행
Analyst Agent	요구사항 분석, 구현 범위 정리, API/화면 계약 초안 작성
Backend Agent	Spring Boot API, Entity, Repository, Service, Controller, 테스트 구현 보조
Frontend Agent	React 화면, API 타입, TanStack Query hook, UI 컴포넌트 구현 보조
Code Reviewer Agent	구현 코드의 버그 가능성, 요구사항 누락, 테스트 부족 지점 검토
Integration QA Agent	프론트엔드와 백엔드 연동 관점의 데이터 흐름, API 응답 정합성, 화면 동작 검증

프로젝트 초반에는 하네스 플러그인을 이용해 `frontend` · `backend` · `fullstack` · `QA` · `code-review` skill 구성을 만들었습니다. 여기서 하네스는 AI가 작업할 때 참조하는 역할별 지시사항, 검증 기준, 작업 절차를 묶은 개발 보조 환경을 의미합니다.

역할을 분리한 이유는 AI가 한 번에 전체 코드를 생성하도록 두기보다, 요구사항 분석·구현·코드 리뷰·통합 QA를 분리해 일관성을 유지하기 위해서입니다. 버그가 발생하면 같은 실수를 반복하지 않도록 하네스의 지시사항과 검증 기준을 다듬으면서 진행했습니다.

UI 스타일은 `styles.refero.design` 에서 참고한 스타일 방향을 `frontend/docs/DESIGN.md` 에 정리한 뒤, 화면 전반의 색상·카드·상태 태그·대시보드 톤이 최대한 일관되도록 구현했습니다.

AI는 주로 초안 작성·반복 구현·검증 관점 보완에 사용했습니다. 도메인 결정, 최종 API 구조, DB 인덱스 조정, 프론트엔드 화면 구성, 테스트 통과 여부 확인은 실제 코드와 실행 결과를 기준으로 직접 검토했습니다.

AI가 생성한 코드 중 직접 수정한 부분과 이유

실제 화면 배치, 간격, 상태별 표시, 반응형 디테일 같은 UI 마감이나 화면 사용성에 대한 부분은 직접 확인하며 일부 수동으로 조정했습니다.

07 미완성 또는 개선하고 싶은 점

시간이 더 있었다면 추가할 것

- **인증/인가** — 조회자·개발자·관리자 역할별 등록/수정/삭제 권한 제어
- **권한별 대시보드 설정** — 계정/역할별 블록 노출·정렬·수정 권한 설정
- **CI/CD Hook 연동** — GitHub Actions·GitLab CI·Jenkins 배포 이벤트 자동 저장
- **감사 로그** — 배포 상태 변경 이력을 별도 audit table 에 저장
- **테스트 보강** — 서비스·배포 CRUD, 대시보드 집계, 프론트 렌더링 테스트 확대

현재 설계의 한계

- 실제 CI/CD와 자동 연동하지 않아 배포 이력을 수동 등록해야 합니다.
- 인증/인가가 없어 운영 환경 배포에는 권한 제어가 부족합니다.
- 알림 기능이 없어 실패·롤백 시 직접 대시보드를 확인해야 합니다.
- H2 기반이라 운영 DB의 성능 특성·full-text search 를 충분히 반영하지 못합니다.
- `keyword` 검색은 부분 문자열 검색이라 데이터가 많아지면 성능 한계가 있습니다.
- PATCH로는 nullable 필드를 다시 비우는 동작을 지원하지 않습니다(빈 입력은 "변경 없음" 처리).
- 대시보드 집계는 요청 시점 계산이라 데이터가 커지면 캐싱·집계 테이블이 필요할 수 있습니다.